

DEPLOYMENT ARCHITECTURE & RATIONALE

Core Health on AWS

Full structure of the planned AWS deployment — network, compute, data and edge — plus exactly how the frontend and backend each get deployed, and the reasoning behind every major choice, specific to this application rather than generic best practice.

REGION: ap-south-1 COMPUTE: ECS on EC2 IaC: Terraform

DECISIONS ALREADY MADE

EC2, not Fargate — the API's traffic follows clinic hours, not spiky bursts, so owning the instances (plus a Reserved Instance) beats paying Fargate's per-task premium.

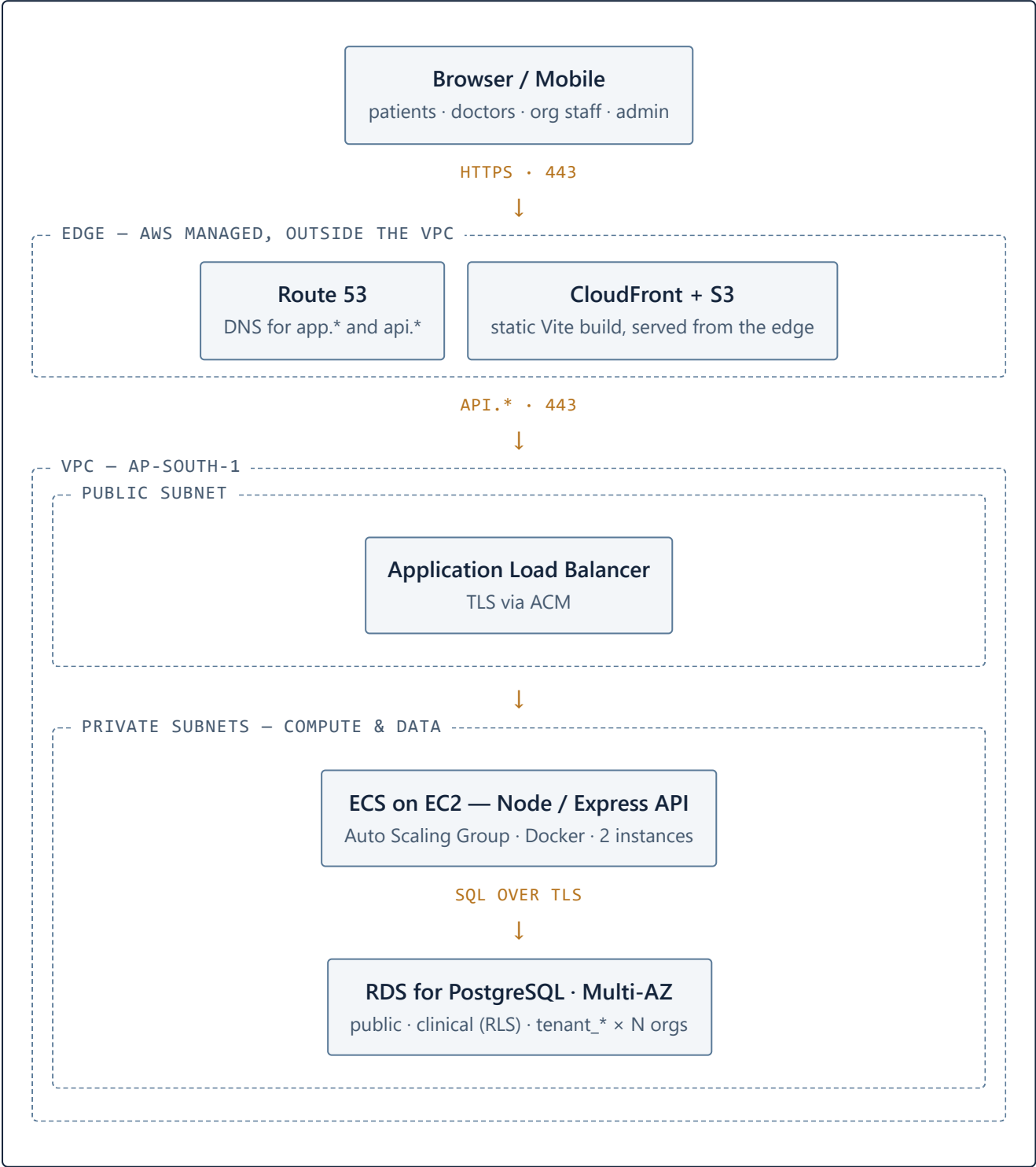
Docker either way — the same image, the same ECR repo, the same task definition shape Fargate would have used; switching compute changed nothing about how the app is packaged.

S3 + CloudFront for the frontend — it's a static Vite/React build with no server-side rendering, so it needs a file host and a CDN, not a running server.

One RDS instance for every tenant — multi-tenancy is already solved in the Postgres schema (`public` `clinical` `· RLS` `tenant_*`), not in the infrastructure, so it doesn't need duplicating per hospital or clinic.

1 Architecture, end to end

The full request path, from a browser to the database, and the network boundary each piece sits inside.

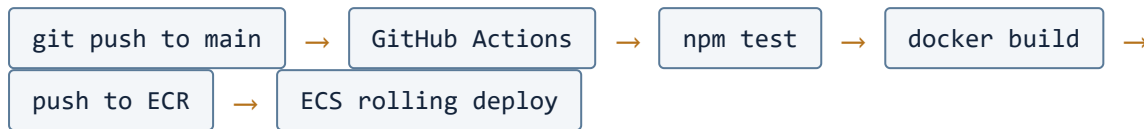


COMPONENT	WHAT IT DOES HERE	WHY THIS ONE, FOR CORE HEALTH SPECIFICALLY
-----------	-------------------	--

Route 53	Resolves app.* and api.* to the right AWS resource	Standard DNS; no app-specific reason beyond needing a domain at all.
CloudFront + S3	Hosts and caches the built React app at edge locations worldwide	The frontend is a static bundle — no server needed. CloudFront gives free HTTPS, DDoS protection, and serves users near-instantly without a round trip to Mumbai every time.
ALB	Public entry point for API traffic, routes to the EC2 fleet	Only one thing in the whole stack is allowed to be hit directly from the internet into the VPC — this is it.
ECS on EC2	Runs the Dockerized Express API, autoscaled across 2+ instances	Steady, predictable (clinic-hours) traffic makes owned EC2 cheaper than Fargate's per-task premium. The app also keeps a persistent Postgres connection pool for the RLS SET LOCAL app.user_id/role/org_id pattern — a long-lived process suits that better than a cold-starting one.
RDS PostgreSQL	One Postgres instance, every tenant's data in it	Tenancy already lives in the schema (tenant_* per org). Running a second database per hospital would duplicate work the schema design already does for free.

2 Backend deploy

How a code change in server/ reaches production.



WHY DOCKER

The API depends on native libraries (the canvas package, used for report rendering) that must match the exact OS they'll run on. Packaging the app as a Docker image — built once in CI on the same base image it'll run on — removes "works on my machine" entirely: the image that passed tests is bit-for-bit the image that runs in production.

WHY ECR, NOT JUST DOCKER HUB

ECR is private by default, lives in the same AWS account and region as the ECS service pulling from it (no public internet hop for the image pull), and the GitHub Actions role (\$5) can be scoped to push to exactly this one repository and nothing else in the account.

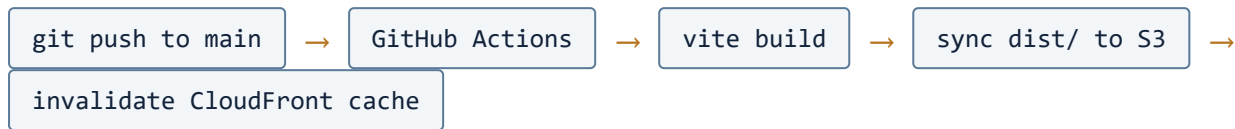
WHY A ROLLING DEPLOY

The ECS service replaces one task at a time — minimum 50% of capacity stays up throughout — so a deploy never drops the API to zero. With 2 instances, that means patients booking appointments never see a gap, even mid-deploy.

The Dockerfile itself is a two-stage build: a **builder** stage with the full compiler toolchain needed to build the canvas native module, and a slim **runtime** stage that copies out only the compiled output and the shared libraries actually needed to run it — smaller image, faster pulls when the Auto Scaling Group scales out.

3 Frontend deploy

How a code change in client/ reaches production — a completely separate, much simpler pipeline from the backend.



WHY NO DOCKER, NO ECS, NO SERVERS HERE

vite build produces plain static files — HTML, JS, CSS. There's nothing to "run." Putting them behind a container/orchestrator would add a whole deploy surface (image, task definition, health checks) to manage a bundle that just needs to sit somewhere and be handed out.

WHY THE CACHE INVALIDATION STEP MATTERS

CloudFront caches aggressively by design — that's the performance win. Without the invalidation step after every sync, users could keep getting yesterday's JavaScript from a nearby edge cache for up to 24 hours after a deploy.

SPA ROUTING NOTE

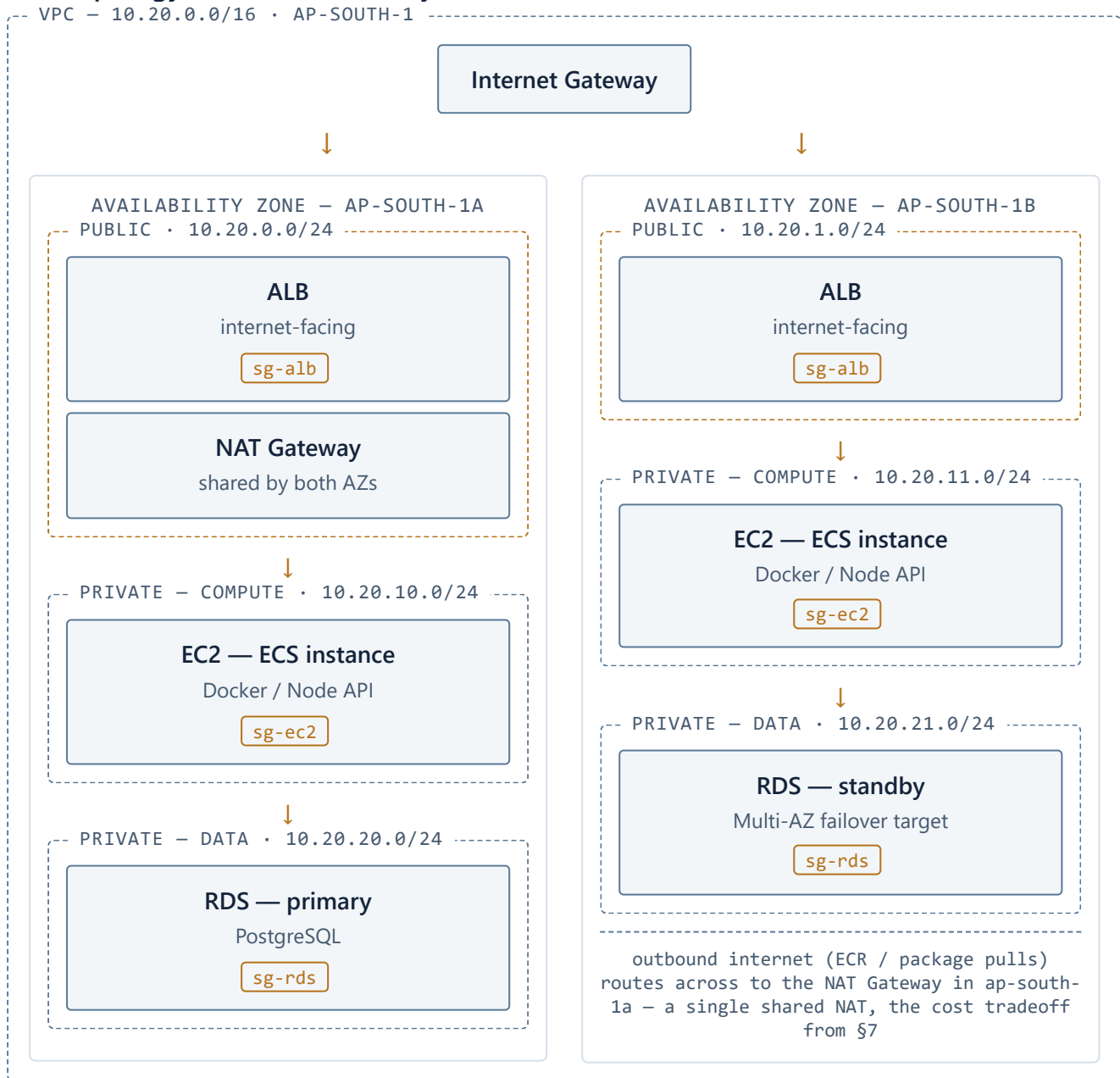
CloudFront is configured to return index.html for any 404, not AWS's default error page. Without that, refreshing the browser on a client-routed page like /patient/dashboard would 404 at the CDN before React Router ever got a chance to handle it.

This pipeline never touches the ECS service, the ALB, or RDS — a typo fix in a label or a CSS tweak ships in roughly the time vite build takes to run, with zero risk to the API's uptime.

4 Network & security

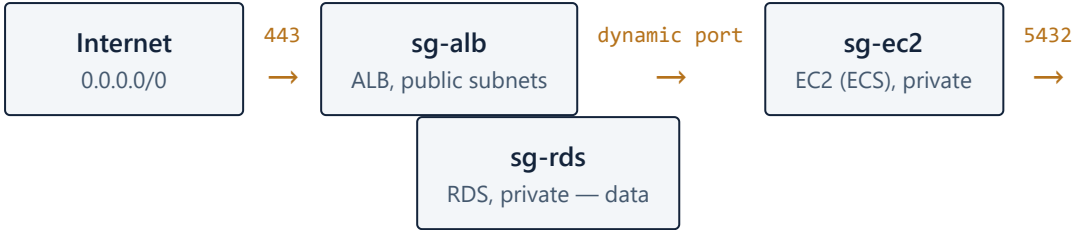
Yes — this runs inside a custom VPC, not the AWS account's default one. Everything below is that VPC's actual shape: which AZ each piece sits in, which subnet, and which security group gates traffic into it.

VPC topology — both Availability Zones



Security group chain — who can talk to whom

Three security groups, each only opening the door to the one directly before it in the request path. Nothing in the data or compute layer is reachable except through this chain.



WHAT THIS BUYS YOU

Even if someone got valid AWS console access scoped only to, say, read S3, they still couldn't reach the database directly — sg-rds only accepts traffic from inside sg-ec2, and sg-ec2 is never reachable from the internet, only from sg-alb. The network enforces the same boundary the IAM roles in the table below enforce — two independent layers, not one.

Subnets — reference table

SUBNET	CIDR	AZ	HOLDS
Public	10.20.0.0/24	ap-south-1a	ALB, NAT Gateway
Public	10.20.1.0/24	ap-south-1b	ALB
Private — compute	10.20.10.0/24	ap-south-1a	EC2 (ECS instances)
Private — compute	10.20.11.0/24	ap-south-1b	EC2 (ECS instances)
Private — data	10.20.20.0/24	ap-south-1a	RDS primary
Private — data	10.20.21.0/24	ap-south-1b	RDS standby (Multi-AZ)

Security groups

SG	INBOUND	OUTBOUND	ATTACHED TO
sg-alb	443 (80 → redirect) from 0.0.0.0/0	to sg-ec2, dynamic port range	ALB
sg-ec2	dynamic port range from sg-alb only	to sg-rds on 5432; 443 out via NAT	EC2 (ASG)
sg-rds	5432 from sg-ec2 only	—	RDS

WHY NO INBOUND SSH ANYWHERE

Shell access to the EC2 instances goes through

SSM SESSION MANAGER

instead — IAM-authenticated and logged, with no open port 22 and no bastion host to maintain or pay for.

IAM roles

ROLE	SCOPE
EC2 instance role	ECS agent registration + SSM Session Manager. Nothing else.
ECS task execution role	Pull from ECR, write CloudWatch Logs, read <i>one specific</i> Secrets Manager secret (the DB credentials) — not a wildcard.
ECS task role	What the app code itself can touch — scoped to the future uploads S3 bucket only (§6).
GitHub Actions deploy role	Federated via OIDC — no stored AWS keys in the repo. Scoped to this one ECR repo, this one ECS service, this one frontend bucket, this one CloudFront distribution.

5 Data — RDS for PostgreSQL

SETTING	START WITH	UPGRADE TO, BEFORE REAL LAUNCH
Instance class	db.t4g.small	size up once real concurrency is known
Multi-AZ	Off — roughly halves the bill pre-launch	On — a booking system can't tolerate an AZ outage during clinic hours
Storage	20 GB gp3, autoscaling to 100 GB	—
Backups	Automated, 7-day retention, deletion protection on	—

The schema itself doesn't change — public / clinical (RLS) / tenant_* move over exactly as defined in corehealth_database.sql. This is a lift of the same database, not a redesign.

6 File uploads — flagged gap

WHY THIS NEEDS FIXING BEFORE GO-LIVE

server.ts currently writes prescription/lab-report uploads to local disk and serves them via express.static. That's harmless on a single Railway instance; on a 2-instance EC2 fleet it breaks silently — a file written on instance A doesn't exist on instance B, and disappears entirely on redeploy.

- 1 Create a private corehealth-uploads S3 bucket (SSE-S3 encryption).
- 2 Swap multer's disk storage for multer-s3 in the upload controllers.
- 3 Serve reports via short-lived presigned URLs instead of express.static.
- 4 Grant the ECS task role read/write on that one bucket only — already provisioned in §4.

7 Rough monthly cost

ITEM	COST
EC2 — 2× t3.small on-demand	~\$34/mo (~\$17–20 with a 1-yr Reserved Instance)
NAT Gateway (single)	~\$33/mo + data processed
ALB	~\$17/mo + LCU usage
RDS db.t4g.small, Single-AZ	~\$26/mo (~\$52 once Multi-AZ)
S3 + CloudFront	~\$1–5/mo at low traffic
Secrets Manager	~\$0.40 / secret / month
Total, pre-launch configuration	roughly \$110–120/month

The equivalent Fargate version of this same diagram runs closer to **\$150–180/month** — the EC2 call is worth roughly \$40–60/month at this scale. Figures are illustrative for ap-south-1; check the AWS Pricing Calculator for current numbers.

8 Build order

- 1 Confirm the Route 53 hosted zone for the real domain (NS delegation done).
 - 2 Request two ACM certificates — one in ap-south-1 for api.* (the ALB), one in us-east-1 for app.* (CloudFront requires this region specifically).
 - 3 Terraform — **network module**: VPC, 6 subnets, route tables, one NAT Gateway, one Internet Gateway.
 - 4 Terraform — **data module**: RDS instance, subnet group, sg-rds.
 - 5 Terraform — **edge module**: S3 frontend bucket, CloudFront + OAC, Route 53 records.
 - 6 Terraform — **ECR module**: one repository for the API image.
 - 7 Build the Docker image once locally and push it to ECR manually — the ECS service needs an image to exist before its first deploy.
 - 8 Terraform — **IAM module**: the four roles from §4.
 - 9 Terraform — **compute module**: launch template, ASG, ALB + target group + listeners, ECS cluster, capacity provider, task definition, service.
 - 10 Put the real DB credentials and JWT secret into Secrets Manager.
 - 11 Point api.* at the ALB and app.* at CloudFront.
 - 12 Wire up the two GitHub Actions workflows (§2, §3) using the OIDC role from step 8.
 - 13 Smoke-test the real domain end to end.
 - 14 Migrate uploads to S3 (§6) — before any real patient file hits this environment.
 - 15 Turn on CloudWatch alarms: ALB 5xx rate, unhealthy host count, RDS CPU/storage, EC2 CPU.
 - 16 Cut DNS over fully; decommission Railway once stable for a few days.
-

Core Health — AWS deployment architecture & rationale. Figures and instance sizes are a starting point, not a fixed prescription.